Estructura del RTL

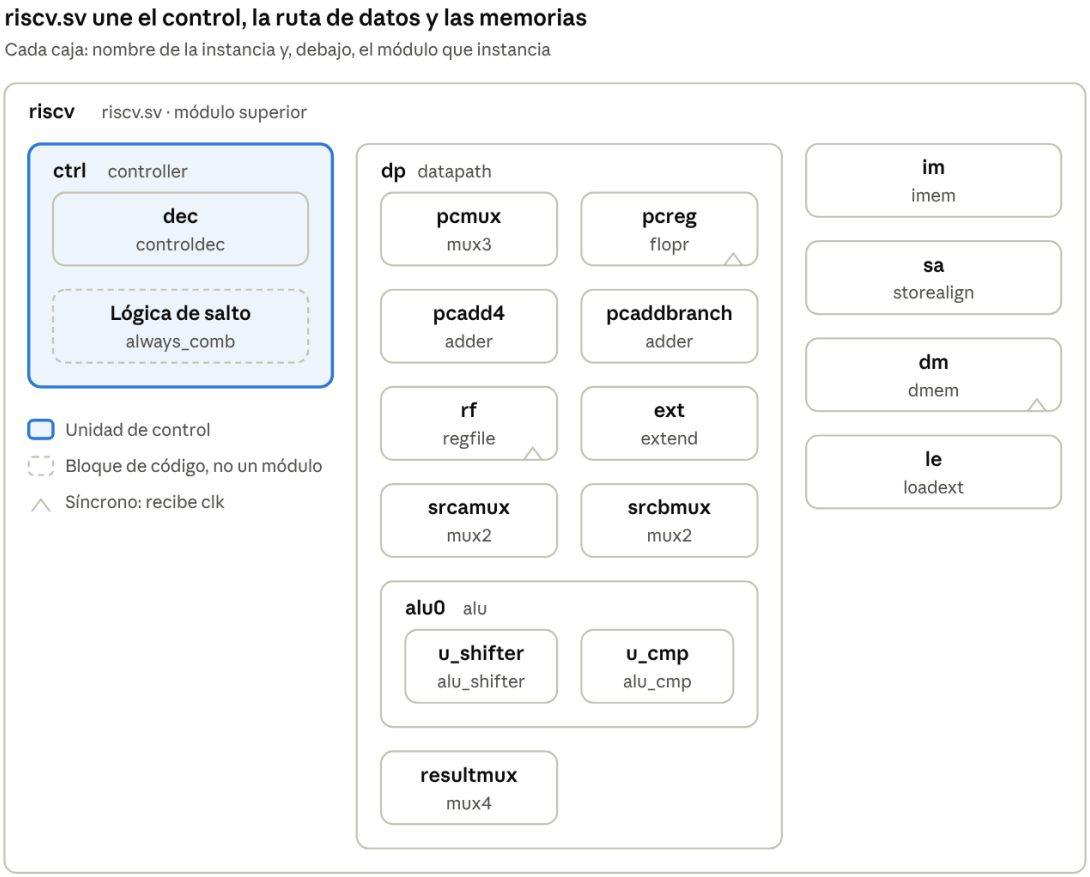
Antonio Rodríguez

1. Introducción

En este documento se intenta explicar la implementación específica del microprocesador RISC-V monociclo que hemos hecho nosotros y que puede diferir ligeramente de otras implementaciones que te puedas encontrar.

El procesador está descrito en 18 módulos de SystemVerilog repartidos en 16 archivos. `riscv.sv` es el módulo superior y une tres bloques: la unidad de control (`controller`), la ruta de datos (`datapath`) y las memorias con sus adaptadores de acceso (`imem`, `dmem`, `storealign` y `loadext`).

2. Jerarquía de módulos



Jerarquía de módulos · 19 instancias bajo riscv

El árbol tiene cuatro niveles: `riscv` contiene seis instancias; `controller` contiene `controldec`; `datapath` contiene diez instancias, y `alu` contiene sus dos módulos auxiliares. Varios módulos se usan más de una vez: `adder` y `mux2` tienen dos instancias cada uno.

3. Resumen de módulos

Cada módulo con su archivo, el nombre de sus instancias (entre paréntesis, el módulo que las contiene) y si es combinacional o tiene parte síncrona.

Módulo	Archivo	Instancias	Tipo	Función
`riscv`	riscv.sv	— (módulo superior)	Estructural	Une control, ruta de datos y memorias
`controller`	controller.sv	`ctrl` (riscv)	Combinacional	Unidad de control: decodificador y lógica de salto
`controldec`	controldec.sv	`dec` (controller)	Combinacional	Traduce la instrucción en señales de control
`datapath`	datapath.sv	`dp` (riscv)	Estructural	Ruta de datos: PC, registros, inmediato, ALU y multiplexores
`flopr`	flopr.sv	`pcreg` (datapath)	Síncrono	Registro del PC
`adder`	adder.sv	`pcadd4`, `pcaddbranch` (datapath)	Combinacional	Calcula PC + 4 y PC + ImmExt
`mux3`	mux3.sv	`pcmux` (datapath)	Combinacional	Elige el siguiente PC
`regfile`	regfile.sv	`rf` (datapath)	Escritura síncrona, lectura combinacional	Banco de 32 registros
`extend`	extend.sv	`ext` (datapath)	Combinacional	Genera el inmediato de 32 bits
`mux2`	mux2.sv	`srcamux`, `srcbmux` (datapath)	Combinacional	Eligen los operandos de la ALU
`alu`	alu.sv	`alu0` (datapath)	Combinacional	Operaciones aritméticas, lógicas y `Zero`
`alu_shifter`	alu.sv	`u_shifter` (alu)	Combinacional	Desplazamientos sll, srl, sra
`alu_cmp`	alu.sv	`u_cmp` (alu)	Combinacional	Comparaciones slt, sltu
`mux4`	mux4.sv	`resultmux` (datapath)	Combinacional	Elige el valor que se escribe en rd
`imem`	imem.sv	`im` (riscv)	Combinacional (solo lectura)	Memoria de instrucciones

Módulo	Archivo	Instancias	Tipo	Función
`storealign`	storealign.sv	`sa` (riscv)	Combinacional	Prepara el dato y la máscara de sb, sh, sw
`dmem`	dmem.sv	`dm` (riscv)	Escritura síncrona, lectura combinacional	Memoria de datos
`loadext`	loadext.sv	`le` (riscv)	Combinacional	Extrae y extiende el dato de lb, lh, lw, lbu, lhu

4. riscv.sv: el módulo superior

`riscv` no contiene lógica propia: solo declara señales internas e instancia y conecta seis bloques. Tiene un parámetro, `INIT_FILE` (por defecto `"mem/imem.hex"`), que pasa a `imem` para indicar qué programa cargar.

Puertos. Entradas `clk` y `rst_n`. Las salidas `PC`, `Instr`, `DataAdr`, `WriteData` y `MemWrite` no las necesita el procesador; están para que el testbench pueda observar lo que ocurre.

Conexiones principales.

- `controller` recibe `Instr` y `Zero`, y entrega a `datapath` las señales `RegWrite`, `ALUSrc`, `ASrc`, `PCSrc`, `ResultSrc` y `ALUControl`.
- `MemWrite` va del `controller` directamente a `dmem`, sin pasar por la ruta de datos.
- `datapath` entrega `PC` a `imem`, y recibe de ella `Instr`.
- La salida `ALUResult` de `datapath` se llama `DataAdr` en este nivel: es la dirección de la memoria de datos.
- `WriteData` (el contenido de rs2) va a `storealign`, que entrega a `dmem` el dato alineado (`MemWriteData`) y la máscara `ByteEn`.
- La palabra leída de `dmem` (`MemReadData`) pasa por `loadext`, y el resultado (`ReadData`) vuelve a `datapath`.
- `storealign` y `loadext` reciben `Instr[14:12]` (funct3) y `DataAdr[1:0]` directamente, sin señales del control.

Que las memorias, `storealign` y `loadext` estén fuera de `datapath` separa el núcleo del sistema de memoria: la ruta de datos solo ve una dirección, un dato de salida y un dato de entrada.

5. Unidad de control: controller.sv y controldec.sv

La unidad de control son dos módulos combinacionales, uno dentro del otro. Su funcionamiento se explica en detalle en la primera parte de este documento, «Unidad de control del RISC-V RV32I monociclo»; aquí se resume cómo está organizado el código.

5.1. controller

Puertos. Entradas `instr[31:0]` y `Zero`. Salidas `RegWrite`, `MemWrite`, `ALUSrc`, `ASrc`, `PCSrc[1:0]`, `ResultSrc[1:0]` y `ALUControl[3:0]`.

Contenido. Tres partes:

1. La instancia `dec` de `controldec`, que genera casi todas las salidas y las señales internas `Branch`, `Jump` y `Jalr`.
2. Un `always_comb` con un `case` sobre `instr[14:12]` que calcula `Taken`: si se cumple la condición del salto, según `funct3` y `Zero`. Es la «lógica de salto» del esquema, que no es un módulo aparte.
3. Una asignación continua: `PCSrc = Jalr ? 2'b10 : {1'b0, Jump | (Branch & Taken)}`.

5.2. controldec

Puertos. Entrada `instr[31:0]`, de la que solo lee `instr[6:0]`, `instr[14:12]` e `instr[30]`. Salidas `RegWrite`, `MemWrite`, `Branch`, `Jump`, `Jalr`, `ResultSrc`, `ALUSrc`, `ASrc` y `ALUControl`.

Contenido. Un único `always_comb`. Primero pone todas las salidas a 0 y después, con un `case` sobre el opcode, activa solo lo que necesita cada clase de instrucción. Tipo R y tipo I aritmética comparten rama y se distinguen por `instr[5]`. Un opcode desconocido no entra en ninguna rama y se ejecuta como un `nop`.

6. Ruta de datos: datapath.sv

`datapath` es estructural: conecta diez instancias y tiene una sola línea de lógica propia, la que forma `JalrTarget`.

Puertos.

- Entradas: `clk`, `rst_n`, las señales de control (`ResultSrc`, `PCSrc`, `ALUSrc`, `ASrc`, `RegWrite`, `ALUControl`), la instrucción `Instr` y el dato leído `ReadData`.
- Salidas: `Zero` (al control), `PC` (a `imem`), `ALUResult` (dirección de `dmem`) y `WriteData` (dato a almacenar).

Señales internas. `PCNext`, `PCPlus4`, `PCTarget`, `JalrTarget`, `ImmExt`, `RegA`, `SrcA`, `SrcB` y `Result`.

Organización. El código está dividido en cinco bloques, en el mismo orden en que circula la información:

1. **PC**: `pcmux` elige `PCNext` entre `PCPlus4`, `PCTarget` y `JalrTarget`; `pcreg` lo guarda; `pcadd4` calcula PC + 4 y `pcaddbranch` calcula PC + ImmExt. `JalrTarget` se forma con `assign JalrTarget = {ALUResult[31:1], 1'b0}`.
2. **Banco de registros**: `rf` lee rs1 (`Instr[19:15]`) y rs2 (`Instr[24:20]`) y escribe `Result` en rd (`Instr[11:7]`).
3. **Extensión del inmediato**: `ext` recibe `Instr` y produce `ImmExt`.
4. **ALU**: `srcamux` elige `SrcA` (rs1 o PC), `srcbmux` elige `SrcB` (rs2 o ImmExt), y `alu0` calcula `ALUResult` y `Zero`.
5. **Resultado**: `resultmux` elige `Result` entre `ALUResult`, `ReadData`, `PCPlus4` e `ImmExt`.

La salida `rs2_data` del banco de registros se llama directamente `WriteData` y se usa dos veces: como entrada 0 de `srcbmux` y como dato que sale hacia `storealign`.

7. Bloques de la ruta de datos

7.1. flopr — registro del PC

Registro de tipo D con ancho parametrizable (`WIDTH`, 32 por defecto). Se actualiza en el flanco de subida de `clk` y tiene reset asíncrono activo a nivel bajo: `always_ff @(posedge clk or negedge`

`rst_n`). Con `rst_n` = 0 la salida pasa a 0, así que el programa empieza siempre en la dirección 0. Su única instancia es `pcreg`.

7.2. adder — sumador

Sumador combinacional de `WIDTH` bits: `assign sum = a + b`. No tiene acarreo de salida. Hay dos instancias: `pcadd4` (PC + 4) y `pcaddbranch` (PC + ImmExt, el destino de los saltos condicionales y de `jal`). Que este segundo sumador exista permite que la ALU compare en el mismo ciclo en que se calcula el destino del salto.

7.3. mux2, mux3 y mux4 — multiplexores

Tres multiplexores combinacionales con ancho parametrizable:

Módulo	Selector	Salida	Instancias
`mux2`	`sel2` (1 bit)	`y2` = `sel2 ? d1 : d0`	`srcamux`, `srcbmux`
`mux3`	`sel3` (2 bits)	`y3` = d0 (00), d1 (01), d2 (1x)	`pcmux`
`mux4`	`sel4` (2 bits)	`y4` = d0 (00), d1 (01), d2 (10), d3 (11)	`resultmux`

En `mux3` el bit alto tiene prioridad: el código 11, que el control nunca genera, también elegiría d2.

7.4. regfile — banco de registros

Array de 32 registros de 32 bits con dos puertos de lectura y uno de escritura.

- **Lectura combinacional**: `rs1_data` y `rs2_data` muestran el registro direccionado en cada momento; si la dirección es 0 devuelven 0, que es lo que define x0.
- **Escritura síncrona**: en el flanco de subida de `clk`, si `rd_write_en` = 1 y `rd_addr` ≠ 0, se guarda `rd_data`. Las escrituras en x0 se ignoran.
- **Reset asíncrono**: con `rst_n` = 0 los 32 registros pasan a 0.

Como la lectura es combinacional y la escritura ocurre al final del ciclo, una instrucción como `add x1, x1, x2` lee el valor antiguo de x1 y escribe el nuevo sin conflicto.

7.5. extend — extensión del inmediato

Recibe la instrucción completa y, con un `case` sobre el opcode, monta el inmediato de 32 bits en el formato que corresponde: I (`0010011`, `0000011`, `1100111`), S (`0100011`), B (`1100011`), U (`0110111`, `0010111`) o J (`1101111`). Todos extienden el signo desde `instr[31]` salvo U, que rellena con ceros los 12 bits bajos. Para cualquier otro opcode, incluido tipo R, da 0. La figura de formatos de la primera parte muestra qué bits toma en cada caso.

8. ALU: alu, alu_shifter y alu_cmp

El archivo `alu.sv` contiene tres módulos. Los dos auxiliares están escritos antes que `alu` para que el compilador los conozca cuando se instancian.

8.1. alu

Puertos. Entradas `a`, `b` (32 bits) y `alucontrol[3:0]`. Salidas `result` (32 bits) y `zero`.

Contenido. Instancia los dos módulos auxiliares, que trabajan siempre en paralelo, y un `always_comb` cuyo `case` sobre `alucontrol` elige el resultado:

- Suma, resta, AND, OR y XOR se calculan directamente en el `case`.
- Para `slt` y `sltu` toma la salida de `u_cmp`.
- Para `sll`, `srl` y `sra` toma la salida de `u_shifter`.
- Cualquier otro código da 0xDEADBEEF, un valor fácil de reconocer al depurar.

`zero` se obtiene con `assign zero = (result == 32'h0)`. La tabla completa de códigos está en la sección 5 de la primera parte.

8.2. alu_shifter

Desplazador combinacional. Recibe `a`, la cantidad `shamt` (conectada a `b[4:0]`) y `alucontrol`:

- 1001: `a << shamt` (sll, slli)
- 1010: `a >> shamt`, entran ceros (srl, srli)
- 1011: `$signed(a) >>> shamt`, entra el bit de signo (sra, srai)
- Cualquier otro código: 0

Como `shamt` son los 5 bits bajos de `b`, sirve igual para las versiones con registro y con inmediato.

8.3. alu_cmp

Comparador combinacional. Recibe `a`, `b` y `alucontrol`:

- 0101: 1 si `$signed(a) < $signed(b)`, si no 0 (slt, slti, blt, bge)
- 1100: 1 si `a < b` sin signo, si no 0 (sltu, sltiu, bltu, bgeu)
- Cualquier otro código: 0

El resultado ocupa 32 bits: `{31'b0, comparación}`.

9. Memorias y acceso a datos

Las dos memorias trabajan con palabras de 32 bits. `storealign` y `loadext` adaptan esas palabras a los accesos de byte y media palabra.

9.1. imem — memoria de instrucciones

Array de 64 palabras (256 bytes) que se carga al empezar la simulación con `initial $readmemh(INIT_FILE, RAM)`. Es de solo lectura y combinacional: `instr = RAM[addr[31:2]]`. Los dos bits bajos del PC se ignoran porque las instrucciones están alineadas a 4 bytes.

9.2. dmem — memoria de datos

Array de 64 palabras (256 bytes).

- **Lectura combinacional** de la palabra completa: `read_data = RAM[addr[31:2]]`. Elegir el byte o la media palabra le toca a `loadext`.
- **Escritura síncrona** en el flanco de subida de `clk`, solo si `mem_write_en` = 1, y solo de los bytes con `byte_en[i]` = 1.
- No tiene reset ni inicialización: una posición vale X hasta que se escribe.

9.3. storealign — alineación de almacenamientos

Combinacional. Recibe `funct3`, los dos bits bajos de la dirección y el contenido de rs2. Con un `case` sobre `funct3[1:0]`:

- `sb`: replica el byte bajo en las cuatro posiciones y activa un solo bit de la máscara, `byte_en = 4'b0001 << addr`.
- `sh`: replica la media palabra baja y activa 0011 o 1100 según `addr[1]`.
- `sw` (y cualquier otro valor): pasa la palabra tal cual con máscara 1111.

Replicar el dato evita desplazarlo: el byte ya está en todas las posiciones y la máscara decide cuál se escribe.

9.4. loadext — extensión de cargas

Combinacional. Recibe `funct3`, los dos bits bajos de la dirección y la palabra leída. Primero selecciona el byte (`mem_data[8*addr +: 8]`) y la media palabra (mitad alta si `addr[1]` = 1); después, según `funct3`, extiende con signo (`lb`, `lh`), con ceros (`lbu`, `lhu`) o deja la palabra completa (`lw` y cualquier otro valor).

10. Reloj, reset y elementos síncronos

Solo tres módulos reciben `clk`: `flopr` (el PC), `regfile` y `dmem`. Todo lo demás es combinacional, así que durante un ciclo la información recorre el procesador sin pararse:

1. El PC sale de `pcreg` y `imem` entrega la instrucción.
2. `controller`, `regfile` y `extend` la decodifican, leen los registros y generan el inmediato.
3. Los multiplexores de operandos y la ALU calculan el resultado o la dirección.
4. `dmem` y `loadext` entregan el dato leído, y `resultmux` elige `Result`.
5. `pcmux` elige el siguiente PC.

En el flanco de subida de `clk` se guardan a la vez el nuevo PC, el resultado en rd (si `RegWrite` = 1) y el dato en memoria (si `MemWrite` = 1). El periodo de reloj debe ser mayor que el camino más largo de este recorrido, que es el de una carga (`lw`): pasa por todos los bloques, incluidos `dmem` y `loadext`.

El reset `rst_n` es asíncrono y activo a nivel bajo. Pone a 0 el PC y los 32 registros; `dmem` no se reinicia, e `imem` conserva el programa cargado al inicio de la simulación.